

บทที่ 5

โครงสร้างและการทำงานของไอพีเทเบิลส์

ลินุกซ์สามารถใช้งานเป็นไฟร์วอลล์ได้ตั้งแต่เคอร์เนล 1.1 ซึ่งเป็นเวอร์ชันแรก โดยอลัน ค็อกซ์ (Alan Cox) ใช้ชื่อว่า ipfw (จาก BSD) ต่อมา ลินุกซ์ 2.0 ได้ถูกพัฒนาและปรับปรุงได้เครื่องมือที่มีชื่อว่า ipfwadm โดยเครื่องมือชิ้นนี้นื้ออนุญาตให้ผู้ใช้สามารถควบคุมกฎการกรองแพ็กเก็ตได้ และต่อมา ลินุกซ์ 2.2 ก็ได้สร้างเครื่องมือตัวใหม่ชื่อ ไอพีเชนส์ (ipchains) ซึ่งเผยแพร่ในปี 1998 โดยรัสตี้ รัสเซล (Rusty Russel) และทีมงาน ทั้งนี้ไอพีเชนส์นี้ถือได้ว่าเป็นพัฒนาการขั้นที่สามของลินุกซ์ไฟร์วอลล์ จวบจนกระทั่งในปัจจุบัน ก็มีเน็ตฟิลเตอร์และ ไอพีเทเบิลส์ซึ่งถือว่าเป็นพัฒนาการขั้นที่สี่ของ ลินุกซ์ ไฟร์วอลล์

เน็ตฟิลเตอร์นั้นเป็นชื่อใหม่ของโค้ดที่ทำหน้าที่เป็น แพ็กเก็ต handler (stateful inspection) ใน ลินุกซ์ เคอร์เนล 2.4 (ที่จริงคือเวอร์ชัน 2.3.15 และเวอร์ชันต่อๆ มา) ซึ่งได้ถูกออกแบบและปรับปรุงใหม่จากเวอร์ชันก่อนหน้านี้ เป็นเรื่องที่น่ายินดีคือเน็ตฟิลเตอร์นั้นสามารถทำงานย้อนหลังร่วมกับไอพีเชนและ ipfwadm ได้ และคำสั่งในการเรียกใช้งานคือ ไอพีเทเบิลส์

ความแตกต่างระหว่างไอพีเทเบิลส์ และ ไอพีเชนส์

- ชื่อของ built-in chain (ประกอบไปด้วย INPUT, OUTPUT, FORWARD) เปลี่ยนจากตัวอักษรเล็ก (lowercase) เป็นตัวอักษรใหญ่ (uppercase)
- การใช้งานที่ต้องระบุพอร์ต ทั้ง ทีซีพี และ ยูดีพี นั้น ต้องใช้คำว่า --source-port หรือ --sport (--destination-port หรือ --dport) และต้องใช้ตามหลังจาก -p tcp หรือ -p udp
- TCP -syn flag เปลี่ยนเป็น --syn และต้องใช้ร่วมกับ -p tcp
- target จาก DENY เปลี่ยนเป็น DROP
- chain ที่ไม่มี กฎ ใดๆ เลยก็สามารถทำงานได้
- การทำ zeroing built-in chain จะทำให้ byte counter ถูกล้างค่าไปด้วย
- ชื่อของ chain ยาวสูงสุดได้ 31 ตัวอักษร
- MASQ เปลี่ยนเป็น MASQUERADE และมีรูปแบบการใช้งานเปลี่ยนไป รวมทั้ง REDIRECT ก็มีการเปลี่ยนแปลงรูปแบบใหม่

ดาวน์โหลด

ก่อนที่ใช้งานไฟร์วอลล์ตัวใหม่นี้ได้ ให้ล็อกอินเป็น root และทดลองพิมพ์คำว่า iptables ว่าคำสั่งนี้มีอยู่หรือไม่ ถ้าไม่มี ต้องดาวน์โหลดจาก <http://www.netfilter.org> โดยให้ดาวน์โหลดเวอร์ชันล่าสุดมา ในที่นี้คือ iptables-1.2.9.tar.bz2 จากนั้นให้ขยายไฟล์และติดตั้งคำสั่งด้านล่าง

```
tar -jxvf iptables-1.2.9.tar.bz2
```

```
cd iptables-1.2.9
```

```
make
```

```
make install
```

ปรับแต่งเคอร์เนล

หลังจากที่มีคำสั่ง iptables อยู่ในระบบแล้ว บางระบบที่มีเคอร์เนลต่ำกว่า 2.4 จำเป็นต้องคอมไพล์เคอร์เนลใหม่ เพื่อให้ใช้งาน iptables ได้ ทั้งนี้โครงการนี้ได้พัฒนาโดยใช้เคอร์เนลเวอร์ชัน 2.4.23 ปัจจุบันซึ่ง

สามารถดาวน์โหลดได้จาก <http://www.kernel.org>

หลังจากที่ดาวน์โหลดมาแล้วให้ขยายไฟล์ไปไว้ที่ /usr/src/linux จากนั้นใช้คำสั่ง make menuconfig หรืออาจใช้ make xconfig ในกรณีที่ติดตั้ง XWindows ไว้แล้ว โดยต้องมั่นใจว่าได้ enable option ต่างๆ ด้านล่างนี้ ภายใต Networkig options

<*> Packet socket

[*] Network packet filtering (replaces ipchains)

<*> Unix domain sockets

[*] TCP/IP networking

[*] IP: advanced router

[*] IP: policy routing

[*] IP: use netfilter MARK value as routing key

[*] IP: fast network address translation

[*] IP: use TOS value as routing key

และภายใต้เมนู "IP: Netfilter Configuration -- >" ให้ enable ทุกออพชันเพื่อให้ใช้งาน netfilter ได้เต็มประสิทธิภาพ

แต่มีอีกจุดหนึ่งที่ห้าม enable ภายใต "Networking options" คือ

[] IP: TCP Explicit Congestion Notification support

เพราะออพชันนี้ จะทำให้บาง แพ็กเก็ต ที่ออกจาก สลินุกซ์ ถูกเซตบิต ECN ไปด้วย ซึ่งจะทำให้ไม่สามารถสื่อสารกับ internet router บางตัวได้

จากนั้นก็ให้คอมไพล์และติดตั้งเคอร์เนลโดยใช้คำสั่งดังนี้

คำสั่งที่ใช้ในการคอมไพล์เคอร์เนล

```
make dep
```

```
make clean
```

```
make bzImage
```

```
make modules
```

#รันคำสั่งนี้ในกรณีที่คอมไพล์เคอร์เนลแบบเลือก module ด้วย

```
make modules_install
```

#รันคำสั่งนี้ในกรณีที่คอมไพล์เคอร์เนลแบบเลือก module ด้วย

```
cp /usr/src/linux/arch/i386/boot/bzImage /boot/bzimage-2.4.23
```

#การ copy bzImage นั้น ขึ้นอยู่กับสถาปัตยกรรมของเครื่องคอมพิวเตอร์นั้นๆ ซึ่งในที่นี้ใช้ i386

```
cp /usr/src/linux/System.map /boot/System.map-2.4.23
```

```
ln -s /boot/System.map-2.4.23 .boot/System.map
```

#ถ้าได้รับ error ให้ลบไฟล์ /boot/System.map ที่ก่อน

จากนั้นให้แก้ไข /etc/lilo.conf เพื่อเพิ่ม configuration ใหม่ ตัวอย่างด้านล่างแสดงเรดแฮท (Red Hat) ลิ눅ซ์ ที่มีเคอร์เนลเดิมเป็น 2.4.18 และเคอร์เนลใหม่เป็น 2.4.23

```
boot=/dev/hda
```

```
map=/boot/map
```

```
install=/boot/boot.b
```

```
prompt
```

```
timeout=50
```

```
message=/boot/message
```

```
default=linux-2.4.23
```

```
image=/boot/vmlinuz-2.4.23
```

```
label=linux-2.4.23
```

```
read-only
```

```
root=/dev/hda1
```

```
image=/boot/vmlinuz-2.4.18
```

```
label=linux-2.4.18
```

```
read-only
```

```
root=/dev/hda1
```

หลังจากนั้นรันคำสั่ง `lilo -v` และรีบูตใหม่ ถ้าไม่มีข้อผิดพลาดใดๆ ก็จะได้เคอร์เนลของ ลินุกซ์ เวอร์ชันล่าสุดที่สนับสนุนการทำงานของ iptables อย่างเต็มที่

5.1 รูปแบบการใช้งาน iptables เบื้องต้น

iptables จะมีรูปแบบการใช้งานดังนี้คือ

```
iptables [table] <command> <match> <target/jump>
```

โดย กฎ ที่เขียนขึ้นจะเป็นเป็นตัวบอกเคอร์เนลว่าให้กระทำ action อย่างไร ในกรณีที่พบ แพ็กเก็ต ตรงตามที่ระบุไว้

- [table] หมายถึง ตารางหรือ table ที่ต้องการระบุ เช่น `iptables -t nat` หมายถึงให้ทำงานกับ nat table
ในกรณีที่ไม่ได้ระบุตาราง iptables จะถือว่าคำสั่งดังกล่าวระบุถึง filter table โดยอัตโนมัติ
- <command> จะเป็นตัวสั่งให้ iptables ทำในสิ่งที่ต้องการ เช่น `iptables -A INPUT` ซึ่งหมายถึงให้สร้าง กฎ ต่อท้าย INPUT chain ใน filter table
- <match> เป็นส่วนที่ใช้ตรวจสอบว่า แพ็กเก็ต มีข้อมูลตรง (match) กับที่ระบุไว้หรือไม่ เช่น มี source ip address เป็น 1.2.3.4
- <target/jump> เป็นตัวระบุว่าจะเจอ แพ็กเก็ต ที่ match ก็จะทำ (action) ตามที่ระบุไว้ เช่น ถ้าแพ็กเก็ต ใดมี source ip address เป็น 1.2.3.4 ให้ DROP แพ็กเก็ต นั้นทิ้งไป

Table

iptables สามารถทำงานได้กับตาราง(table) 3 ตารางหลัก สามารถระบุตารางได้โดยใช้ตัวเลือก -t ตามด้วยชื่อ table คือ

1. Filter table ใช้สำหรับกรอง แพ็กเก็ต มี 3 built-in chain คือ INPUT, OUTPUT, FORWARD ซึ่งจะได้อธิบายรายละเอียดต่อไป
2. Nat table ใช้สำหรับการแปลงแอดเดรส (Network Address Translation) มี 3 built-in chain คือ PREROUTING, POSTROUTING, OUTPUT ซึ่งรายละเอียดจะได้อธิบายต่อไป
3. Mangle table เป็นตารางที่ใช้เปลี่ยนแปลงหรือแก้ไข แพ็กเก็ต เช่น เปลี่ยนค่า TTL, MARK ซึ่งปกติจะใช้ในการทำ routing ที่มีความซับซ้อนสูง มี 2 built-in chain คือ PREROUTING chain (ใช้แก้ไข แพ็กเก็ต ก่อนที่จะเข้าสู่ไฟร์วอลล์และก่อนเข้าสู่ routing decision) และ OUTPUT chain (ใช้แก้ไข แพ็กเก็ต ที่ถูกสร้างโดยไฟร์วอลล์ก่อนที่มันจะถูกส่งไปยัง routing decision)
ทั้งนี้ไม่สามารถทำ network address translation หรือ masquerading ที่ table นี้ได้ และโครงการนี้ไม่ได้ศึกษาในส่วนนี้

Command

- -A เพิ่ม กฎ ใหม่ต่อท้าย chain (Append rule) เช่น
iptables -A INPUT -p ALL -i eth0 -j ACCEPT
- -D ลบ กฎ (Delete rule) เช่น
iptables -D INPUT --dport 80 -j DROP
- -I เพิ่ม กฎ ใหม่ ใน chain (Insert rule) เช่น
iptables -I OUTPUT -p ALL -s 127.0.0.1/32 -j ACCEPT
- -R แทนที่ กฎ เดิม ด้วย กฎ ใหม่ (Replace rule)
- -L แสดง กฎ ทั้งหมดใน chain (ถ้าไม่ระบุ chain จะแสดง กฎ ทั้งหมดใน filter table ทั้งสาม built-in chain) เช่น
iptables -L
iptables -L -t nat
iptables -L INPUT
- -F ลบ กฎ ทั้งหมดใน chain ทิ้ง เช่น
iptables -F INPUT
iptables -F mychain
- -Z ใช้ reset byte counter สำหรับทุก กฎ ใน chain ที่กำหนด เช่น
iptables -Z INPUT
- -N ใช้สร้าง chain ใหม่ เช่น
iptables -N mychain
- -X ลบ chain ที่ไม่มี กฎ ซึ่งสามารถลบ user-defined chain ที่ไม่มี กฎ ได้ แต่ไม่สามารถลบ built-in chain ได้ เช่น
iptables -X emptychain
- -P เปลี่ยน default policy ของ chain ค่าที่ใช้ได้คือ ACCEPT, DROP ทั้งนี้ค่านี้มีความสำคัญอย่างมากเพราะหาก แพ็กเก็ต ถูกส่งเข้ามาใน chain แล้ว และไม่ match กับ กฎ ใดๆ เลย แพ็กเก็ต นั้นก็ต้องถูกตัดสินใจโดย policy ของ chain นั้นๆ เช่น
iptables -P FORWARD DROP

ซึ่งหาก แพ็กเก็ต ถูกส่งเข้ามายัง FORWARD chain และไม่ match กับ กฎ ใดๆ ใน FORWARD chain นี้เลย มันก็จะถูก DROP ทันที

- -E ใช้เปลี่ยนชื่อ chain ใหม่ เช่น

```
# iptables -E myoldchain mynewchain
```

การใช้ command ด้านบนนั้นสามารถใช้ร่วมกับอปชันบางอย่างได้ คือ

- -V, --verbose ใช้ร่วมกับ -L, -A, -I, -D, -R เพื่อให้เห็นจำนวน byte ที่ match กับ กฎ ออกมาด้วย (หน่วยเป็นได้ทั้ง K(x1,000),M(x1,000,000),G(x1,000,000,000)) เช่น
iptables -L -v
- -x, --exact ใช้ร่วมกับ -L และ -v เพื่อให้เห็นจำนวน แพ็กเก็ต และจำนวนของ byte ข้อมูลที่ match โดยไม่แสดงผลในหน่วยของ K,M,G เช่น
iptables -L OUTPUT -v -x
- -n, --numeric ใช้ร่วมกับ -L เพื่อสั่งให้ iptables แสดงข้อมูลไอพีแอดเดรสและ พอร์ต เป็นตัวเลข เท่านั้น เช่น
iptables -L OUTPUT -n
- --line-numbers ใช้ร่วมกับ -L เพื่อแสดงเลขบรรทัดของ กฎ ซึ่งตัวเลขที่แสดงนี้จะสามารถใช้ได้กับคำสั่ง แทรก กฎ ที่ระบุเป็นลำดับที่ของ กฎ เช่น
iptables -L --line-numbers
- --modprobe=command เพื่อโหลด module ที่เกี่ยวข้อง

5.2 ฟังก์ชันแมทช์ (Match)

การตั้งเงื่อนไขของการ match นั้นจะต้องอาศัยความเข้าใจในเรื่อง IP, ทีซีพี, ยูดีพี, และ ไอซีเอ็มพี มาบ้างพอสมควร จึงจะสามารถตั้งเงื่อนไขที่เหมาะสมและตรงตามความต้องการได้ ซึ่งมีรายละเอียดดังนี้

- การระบุ source, destination IP address
สามารถระบุ source ip address ของ แพ็กเก็ต โดยใช้ -s หรือ --source หรือ --src และสำหรับ destination ip address ก็ใช้ -d หรือ --destination หรือ --dst การระบุไอพีแอดเดรสนั้นสามารถทำได้ 4 แบบด้วยกันคือ
 1. ใช้ชื่อเต็มแทน เช่น localhost หรือ www.ce.kmitl.ac.th
 2. ระบุไอพีแอดเดรสโดยตรง เช่น 127.0.0.1 หรือ 161.246.5.14

3. ระบุเป็น group ของไอพีแอดเดรส เช่น 161.246.5/24 ซึ่งหมายถึงไอพีแอดเดรสตั้งแต่ 161.246.5.0 – 161.246.5.255
4. หรืออาจจะใช้ 161.246.5.0/255.255.255.0 แทน 161.246.5.0/24 ได้

- การทำ Inversion

ในบางกรณีนั้นหากต้องการระบุเป็น inverse เช่น อนุญาตให้ทุกไอพียกเว้นไอพีที่ระบุไว้ ซึ่งการใช้คำสั่งดังกล่าวสามารถทำได้โดยใช้เครื่องหมาย ! นำหน้า argument ที่ต้องการ (เครื่องหมาย ! หมายถึง NOT) เช่น -p ! TCP ซึ่งจะ match กับโปรโตคอลทุกๆ ตัวที่ไม่ใช่ ทีซีพี หรือ -s ! localhost ซึ่งหมายถึง แพ็กเก็ต ที่มี source ip address อื่นๆ ยกเว้น localhost (127.0.0.1)

- การระบุโปรโตคอล

สามารถระบุโปรโตคอลที่ต้องการได้ดังนี้คือ ทีซีพี, ยูดีพี, ไอซีเอ็มพี หรือสามารถใช้ตัวเลขแทนได้ (สำหรับ *NIX อ้างอิงได้จาก /etc/protocols) และยังสามารถใช้ได้ทั้งตัวอักษรเล็กหรือใหญ่ (ใช้ได้ทั้ง tcp และ TCP) เช่น -p TCP หรือ -p ! tcp

- การระบุ interface

-i หรือ --in-interface ตามด้วยชื่อ interface ใช้เพื่อระบุ incoming interface ซึ่งหมายถึงว่า แพ็กเก็ต ที่จะ match กับ กฎ นี้ต้องเข้ามาจาก interface ที่กำหนด เช่น -i eth0 หมายความว่า ทุก แพ็กเก็ต ที่เข้ามาทาง eth0 จะ match กับ กฎ นี้ ทั้งนี้ชื่อ interface ที่สามารถใช้ได้นั้น สามารถตรวจสอบได้โดยใช้คำสั่ง ifconfig

และ -o หรือ --out-interface ตามด้วยชื่อของ interface ใช้เพื่อระบุ outgoing interface ซึ่งหมายถึงว่า แพ็กเก็ต ที่จะ match กับ กฎ นี้ กำลังจะเดินทางผ่าน interface ที่ระบุไว้ เช่น -o eth1 หรือ -o ! eth1

ข้อสังเกต

- o สำหรับ INPUT chain นั้นไม่มี output interface ดังนั้นหากใช้ -o ร่วมกับ INPUT chain ก็จะไม่มีการแพ็กเก็ต ใดที่ match กับ กฎ นี้เลย
- o ทำนองเดียวกันกับ OUTPUT chain ที่ไม่มี input interface ดังนั้นหากใช้ -i ร่วมกับ OUTPUT chain ก็ไม่มีประโยชน์อันใด
- o FORWARD chain มีได้ทั้ง input และ output interface
- o หากระบุ interface ที่ไม่มีอยู่จริง ก็จะไม่มีการแพ็กเก็ต ใดที่ match กับ กฎ นั้นเลย

- o หากใช้เครื่องหมาย + ร่วมกับ interface เช่น `ppp+` นั้นจะหมายถึงทุกๆ `ppp` interface เช่น `ppp0`, `ppp1`

- fragment packet

ในการส่งข้อมูลใน ip network นั้นเป็นเรื่องปกติที่จะเกิดการ fragment ของ แพ็กเก็ต เนื่องจากขนาดของ แพ็กเก็ต มีขนาดใหญ่เกินไปที่จะส่งไปในครั้งเดียว จำเป็นต้องมีการแบ่ง แพ็กเก็ต ออกเป็นหลายๆ ชิ้นทยอยส่งไป ซึ่งเรียกกันว่าการทำ fragment โดยเครื่องปลายทางจะทำหน้าที่ประกอบ fragment packet รวมกันเป็น แพ็กเก็ต ที่สมบูรณ์ดั้งเดิม

ข้อมูลที่เป็น fragment packet นั้นจะมี header ที่สมบูรณ์แค่ แพ็กเก็ต แรกเท่านั้น ตัว แพ็กเก็ต ที่ตามมาจะมีแค่ header บางส่วนคือ ไอพีแอดเดรสเท่านั้น ไม่มีข้อมูลของโปรโตคอลแบบมาด้วย ดังนั้นการตรวจสอบข้อมูล header ของ ทีซีพี, ยูดีพี, ไอซีเอ็มพี จึงไม่สามารถทำได้ใน แพ็กเก็ต ที่สองเป็นต้นมา

หากใช้ NAT บรรดา fragment packet จะถูกประกอบเข้าด้วยกันจนสมบูรณ์ก่อนที่ แพ็กเก็ต จะเข้าไปถึง packet filtering ดังนั้นจึงไม่มีความจำเป็นที่จะต้องกังวลเกี่ยวกับ fragment packet

ดังนั้นถ้าไม่ได้ใช้ NAT ก็ควรทำความเข้าใจไว้ว่า iptables มีกระบวนการในการทำงานกับ fragment packet อย่างไร หลังจากที่ fragment packet แรกผ่านเข้ามาแล้ว iptables สามารถตรวจสอบได้ว่า จะอนุญาตให้ผ่านหรือไม่ ในขณะที่ fragment packet ที่สองและหลังจากนั้นที่ตามมานั้น จะไม่สามารถ match กับ กฎ ใดๆ เลย เช่น `-p TCP --sport www` หรือแม้แต่ `-p TCP --sport ! www`

อย่างไรก็ตาม สามารถเขียน กฎ ให้ตรวจสอบทั้ง fragment packet ตัวที่สองและหลังจากนั้นที่ตามมาได้ด้วยการใช้ `-f` หรือ `--fragment` ทั้งนี้อาจจะเขียนในทางตรงข้ามคือไม่ต้องตรวจสอบ fragment packet ที่สองและหลังจากนั้นโดยใช้ `! -f` ก็ได้

ทั้งนี้โดยปกติแล้วมักจะปล่อยให้ fragment packet ผ่านไป เนื่องจากถ้าสามารถ DROP ตัว fragment packet ตัวแรกได้แล้ว มันก็ไม่สามารถถูกประกอบที่เครื่องปลายทางได้ แต่ทั้งนี้ fragment packet ที่ถูกปล่อยไปดังกล่าวอาจจะทำให้เครื่องที่ได้รับ hang หรือ crash ได้ หรืออาจจะเกิดการโจมตีแบบ Denial of Service โดยใช้ fragment packet ได้

- TCP extension

ถ้ามีการเรียกใช้ `-p tcp` ตัว TCP extension ก็จะถูกโหลดมาใช้งานโดยอัตโนมัติ โดยมีופןให้เลือกใช้งานดังนี้

- o `--tcp-flags mask flags` : mask นั้นหมายถึง flag ที่ต้องการตรวจสอบ และ flag เป็นตัวที่บ่งชี้ว่า flag ใดต้องถูก set บ้าง

เช่น `# iptables -A INPUT -p tcp --tcp-flags ALL SYN,ACK -j DROP`

โดย ALL นั้นหมายถึงทุกๆ flag (SYN,ACK,FIN,RST,URG,PSH) และถ้า flag

SYN,ACK ถูก set พร้อมกันก็ให้ drop packet นั้นทิ้งไป นอกจากนี้ยังสามารถใช้ NONE ซึ่งหมายถึงไม่มี flag ใดถูก set ได้

- o --syn เป็นสัญลักษณ์ย่อของ --tcp-flags SYN,RST,ACK SYN
- o --source-port หรือ --sport สามารถใช้ได้ทั้งตัวเลขและตัวอักษร (อ้างอิงจากไฟล์ /etc/services) และระบุเป็น พอร์ต เดียว หรือช่วงของ พอร์ต ได้
เช่น --sport 21:25 หมายถึง พอร์ต 21 - 25 , --sport 25: หมายถึงพอร์ตที่มากกว่าหรือเท่ากับ 25 , --sport :25 หมายถึงพอร์ตที่น้อยกว่าหรือเท่ากับ 25
- o --destination-port หรือ --dport มีรูปแบบการใช้งานเช่นเดียวกันกับ --sport
- o --tcp-option ใช้ตรวจสอบ TCP option ว่าตรงกับเลขที่ระบุไว้หรือไม่

- อธิบาย flag ของ ทีซีพี เพิ่มเติม

การเชื่อมต่อโดยใช้ ทีซีพี นั้น ผู้ที่เริ่มสร้าง connection จะเป็นผู้ส่ง SYN แพ็กเก็ตมายังเครื่องปลายทาง ดังนั้นหากไม่ต้องการให้เครื่องใดเป็นผู้เริ่มสร้างการติดต่อก็สามารถ block ไอพีดังกล่าวได้ โดยใช้ --syn เช่น -p TCP -s x.x.x.x --syn หากยังไม่เข้าใจรูปแบบการเชื่อมต่อแบบ ทีซีพี นี้แล้ว ก็เป็นการยากที่จะสร้าง กฎ สำหรับ iptables ดังนั้นจึงขอแนะนำให้ไปศึกษาหลักการทำงานเบื้องต้นของทั้ง ทีซีพี, ยูดีพี, ไอซีเอ็มพี มาก่อน

- UDP extension

คล้ายกันกับ ทีซีพี ตัว ยูดีพี extension มีตัวเลือกให้ใช้เพียงแค่ 2 อย่างเท่านั้นคือ --source-port (--sport) และ --destination-port (--dport) โดยต้องระบุ -p udp ด้วย

- ICMP extension

โดยการระบุ -p icmp ก็สามารถใช้งาน ICMP extension ได้ โดยมีตัวเลือกให้เลือกคือ -icmp-type เช่น --icmp-type host-unreachable (หรือใช้เลข 3 แทนได้) นอกจากนี้ยัง สามารถระบุ type/code ได้ เช่น 3/3 ซึ่งหมายถึง port unreachable

TYPE	CODE	Description	Query	Error
0	0	Echo Reply	x	
3	0	Network Unreachable		x
3	1	Host Unreachable		
3	2	Protocol Unreachable		
3	3	Port Unreachable		
3	4	Fragmentation needed but no frag. bit set		

TYPE	CODE	Description	Query	Error
3	5	Source routing failed		
3	6	Destination network unknown		
3	7	Destination host unknown		
3	8	Source host isolated (obsolete)		
3	9	Destination network administratively prohibited		
3	10	Destination host administratively prohibited		
3	11	Network unreachable for TOS		
3	12	Host unreachable for TOS		
3	13	Communication administratively prohibited by filtering		
3	14	Host precedence violation		
3	15	Precedence cutoff in effect		
4	0	Source quench		
5	0	Redirect for network		
5	1	Redirect for host		
5	2	Redirect for TOS and network		
5	3	Redirect for TOS and host		
8	0	Echo request	x	
9	0	Router advertisement		
10	0	Route solicitation		
11	0	TTL equals 0 during transit		x
11	1	TTL equals 0 during reassembly		
12	0	IP header bad (catchall error)		
12	1	Required options missing		
13	0	Timestamp request (obsolete)		

TYPE	CODE	Description	Query	Error
14	0	Timestamp reply (obsolete)		
15	0	Information request (obsolete)		
16	0	Information reply (obsolete)		
17	0	Address mask request		
18	0	Address mask reply		

ตารางที่ 5-1 แสดงรายละเอียดของ ICMP message

- Match Extension

เป็น netfilter package ที่อยู่ในช่วงทดลองใช้ รูปแบบการใช้งานให้ใช้ -m แล้วตามด้วย match ที่ต้องการ เช่น -m mac ทั้งนี้มีออปชันให้เลือกใช้งานดังต่อไปนี้

- o mac

รูปแบบการใช้งาน: -m mac หรือ --match mac

ใช้ตรวจสอบ source MAC address ว่าตรงกับค่าที่ระบุไว้หรือไม่ มีประโยชน์สำหรับ PREROUTING, INPUT chain โดยมีออปชันให้ใช้งานคือ

- --mac-source เช่น --mac-source 00:55:81:CC:42:FF

- o limit

รูปแบบการใช้งาน: -m limit หรือ --match limit

ใช้เพื่อจำกัดจำนวนของการ match ที่อาจจะมากเกินไป เป็นประโยชน์สำหรับ กฎ ที่วางไว้ตอนท้ายสุดของ chain (ใช้ร่วมกันกับ DROP policy) ซึ่งส่วนใหญ่เป็น กฎ ที่ใช้เก็บข้อมูลลงล็อกไฟล์ ซึ่งถ้าผู้บุกรุกส่ง แพ็กเก็ต ที่ไม่เข้าข่าย กฎ ใดๆ ใน chain จนกระทั่งมาถึง กฎ ที่ทำหน้าที่เก็บล็อกนี้ ถ้า แพ็กเก็ต ที่เข้ามาจำนวนมากก็อาจจะทำให้ฮาร์ดดิสก์เต็มได้ ดังนั้นจึงต้องใช้จำกัดจำนวนในการเก็บข้อมูลลงล็อก ซึ่งมีออปชันให้ใช้งานดังนี้คือ

- --limit ตามด้วยตัวเลข ซึ่งบอกถึงจำนวนครั้งสูงสุดของการ match กับ กฎ ที่ยินยอมต่อ 1 วินาที เช่น --limit 5/s ทั้งนี้ยังสามารถใช้หน่วยเวลาอื่นได้ เช่น /second /minute /hour /day เช่น -m limit --limit 3/minute
 - --limit-burst ตามด้วยตัวเลข แสดงถึงจำนวนมากที่สุดของ แพ็กเก็ต ที่ match กับ กฎ นี้ ค่า default ของมันคือ 5

ตัวอย่างการใช้ --limit และ --limit-burst ร่วมกัน เช่น

```
# iptables -A INPUT -m limit --limit 3/minute --limit-burst 3 -j LOG
```

โดยส่วนใหญ่นิยามกฎ กฎ นี้ไว้เป็น กฎ สุดท้ายใน chain โดยเฉพาะ chain ที่มี default policy เป็น DROP เพื่อเป็นตัวเก็บหลักฐานว่ามี แพ็กเก็ตใดที่ถูกส่งมาและไม่ผ่านการตรวจสอบจาก กฎ และกำลังจะถูก DROP โดย default policy โดย กฎ ด้านบนนี้กำหนดจำนวน match สูงสุดไว้ 3 ครั้งต่อนาที ซึ่งแสดงว่าใน 1 นาทีจะมีการบันทึกล็อกได้สูงสุด 3 ครั้งเท่านั้น และมีค่า burst เท่ากับ 3 ซึ่งอธิบายได้ว่า ถ้าสมมุติมี แพ็กเก็ต ที่ match กับ กฎ นี้ 3 ครั้งภายใน 2 วินาที และถึงแม้ว่าจะมี แพ็กเก็ต ที่ match ส่งมาอีกก็ จะไม่มีการ บันทึกล็อกแต่อย่างใด และจะต้องรอไปอีก 1 นาทีจึงจะมีการเริ่มการบันทึกล็อกใหม่อีกครั้ง ซึ่งมี ประโยชน์ในกรณีที่มีคนต้องการส่ง แพ็กเก็ต เพื่อ flood log หรือทำให้ล็อกในเครื่องเต็ม ทั้งนี้นิยมใช้ ร่วมกับ --log-level (อ้างอิงค่าจาก level ใน syslogd เพื่อกำหนดค่า level สำหรับ syslog) และ --log-prefix เพื่อใช้อธิบายเพิ่มเติม เช่น

```
# iptables -A INPUT -m limit --limit 3/minute --limit-burst 3 -j LOG --log-level DEBUG --log-prefix
"Packet died: "
```

นอกจากนี้ยังใช้ป้องกันการโจมตีแบบ Denial of Service เช่น SYN flood ได้ด้วย เช่น

```
# iptables -A FORWARD -p tcp --syn -m limit --limit 1/s -j ACCEPT
```

ใช้ป้องกันการโจมตีแบบ Ping of Death

```
#iptables -A FORWARD -p icmp --icmp-type echo-request -m limit --limit 1/s -j ACCEPT
```

โดยปกติมักใช้วิธี drop ไอซีเอ็มพี แพ็กเก็ต ทั้งหมด เพราะถือว่า ไอซีเอ็มพี แพ็กเก็ต เป็นข้อมูลที่ มีอันตราย และสามารถปลอมแปลงได้ง่าย

หรือใช้ป้องกันการถูก scan

```
# iptables -A FORWARD -p tcp --tcp-flags SYN,ACK,FIN,RST RST -m limit --limit 1/s -j ACCEPT
```

โดยปกติไม่นิยมใช้วิธีนี้นัก เพราะมีทางเลือกที่ดีกว่า คือการตรวจสอบจาก state match ว่าเป็นการเชื่อมต่อใหม่หรือไม่ (state = new) ถ้าใช่และ SYN bit ไม่ถูก set ตัว แพ็กเก็ต นั้นก็จะถูก DROP ทิ้งไป

O owner

รูปแบบการใช้งาน: -m owner หรือ --match owner

ใช้ตรวจสอบลักษณะของ แพ็กเก็ต ว่าใครเป็นผู้สร้าง ซึ่งสามารถใช้ได้กับ OUTPUT chain เท่านั้น และใช้ได้กับบาง แพ็กเก็ต ที่มีเจ้าของ เช่น ไอซีเอ็มพี แพ็กเก็ต นั้นใช้ไม่ได้เพราะไม่มีเจ้าของ มีอุปสรรคให้ใช้งานดังนี้คือ

- `--uid-owner userid`

ใช้ตรวจสอบว่า แพ็กเก็ต ถูกสร้างโดย user id ที่ระบุไว้หรือไม่ (ใช้ตัวเลขแทน userid เท่านั้น)

- `--gid-owner groupid`

ใช้ตรวจสอบว่า แพ็กเก็ต ถูกสร้างโดย user ที่อยู่ใน group id ที่ระบุไว้หรือไม่ (ใช้ตัวเลขแทน groupid เท่านั้น)

- `--pid-owner processid`

ใช้ตรวจสอบว่า แพ็กเก็ต ถูกสร้างขึ้นจาก process ที่มี process id ตรงกับที่ระบุไว้หรือไม่

- `--sid-owner sessionid`

ใช้ตรวจสอบว่า แพ็กเก็ต ถูกสร้างโดย process ที่อยู่ใน session group ที่กำหนดไว้หรือไม่

O unclean

รูปแบบการใช้งาน: `-m unclean` หรือ `--match unclean`

เป็นโมดูลที่อยู่ในระหว่างการทดลองใช้งาน นอกจากนี้ยังไม่มีออพชันสำหรับใช้งาน และโปรดระมจะระงับหากจะนำไปใช้กับเครื่องที่ต้องการความปลอดภัย เนื่องจากอาจจะยังมีข้อบกพร่องของโปรแกรมอยู่

โดย แพ็กเก็ต ที่เข้าข่าย unclean คือ

- แพ็กเก็ต ที่มี header ของ ไอซีเอ็มพี/ทีซีพี/ยูดีพี สั้นหรือไม่สมบูรณ์
- ทีซีพี, ยูดีพี แพ็กเก็ต ที่มี source หรือ destination ip address เป็นศูนย์
- ทีซีพี แพ็กเก็ต ที่ใช้ flag ผสมกันแบบผิดปกติ
- แพ็กเก็ต ที่ใช้ ทีซีพี option, IP option เกินความยาวที่กำหนดไว้ หรือมีความยาวของออพชันเป็นศูนย์
- fragment แพ็กเก็ต ที่ไม่สมบูรณ์ ทั้งด้านความยาวและค่า offset ที่เหลื่อมซ้อนกัน เช่น Ping of Death

O multiport ใช้ร่วมกับ `--sport` หรือ `--dport` ในกรณีที่ต้องการระบุ พอร์ต จำนวนมากกว่าหนึ่ง เช่น

`-m multiport -p tcp --sport 25,80,53`

• The State Match

รูปแบบการใช้งาน: `-m state` หรือ `--match state`

เป็นโมดูลที่ใช้ประโยชน์ได้เป็นอย่างดี มีออพชันให้ใช้งานดังนี้

o NEW

รูปแบบการใช้งาน: `-m state --state new` หรือ `--match state --state new`

หมายถึง แพ็กเก็ต ที่เป็นตัวสร้าง connection ใหม่

o ESTABLISHED

รูปแบบการใช้งาน: `-m state --state established` หรือ `--match state --state established`

หมายถึง แพ็กเก็ต ที่เกี่ยวข้องกับ connection ที่สร้างไว้แล้ว เช่น echo-reply แพ็กเก็ต หรือ แพ็กเก็ต ที่ส่งข้อมูลออกไปจาก web server เมื่อมี request web service เข้ามา

o RELATED

รูปแบบการใช้งาน: `-m state --state related` หรือ `--match state --state related`

เป็น แพ็กเก็ต ที่เกี่ยวข้องกับ connection ที่สร้างไว้แล้ว แต่ไม่ใช่ส่วนหนึ่งส่วนใดของ connection นั้น เช่น FTP data แพ็กเก็ต (พอร์ต 20) ที่เกิดขึ้นจากการใช้คำสั่งใน FTP command (พอร์ต 21)

o INVALID

รูปแบบการใช้งาน: `-m state --state invalid` หรือ `--match state --state invalid`

เป็น แพ็กเก็ต ที่ไม่เกี่ยวข้องกับส่วนอื่นเลย เช่น ไอซีเอ็มพี echo-reply ที่เกิดขึ้น โดยที่ไม่มีเครื่องใดในระบบส่ง echo-request ออกไปเลย (กรณีเช่นนี้เกิดขึ้นได้เนื่องจาก อาจจะมี โคมินโจมดีแบบ Smurf attack)

5.3 การระบุการเกิด (target)

เมื่อมี แพ็กเก็ต ที่ match กับ กฎ แล้ว ต้องกำหนด target สำหรับ แพ็กเก็ต ไว้ด้วย โดยปกติจะใช้กัน 2 target คือ DROP และ ACCEPT นอกจากนี้ยังมี target แบบอื่นได้คือ

- user-defined chain

เนื่องจาก iptables อนุญาตให้ผู้ใช้สามารถสร้าง chain ขึ้นมาได้ใหม่นอกเหนือจาก built-in chain ทั้งสามตัว (INPUT, OUTPUT, FORWARD) ทั้งนี้จะต้องใช้ตัวอักษรตัวเล็กทั้งหมดสำหรับ chain ที่ผู้ใช้สร้างขึ้นเอง

เมื่อ แพ็กเก็ต match กับ กฎ ที่เป็น user-defined chain ตัว แพ็กเก็ต จะถูกนำไปตรวจสอบใหม่ โดย user-defined chain นั้นๆ และถ้าใน chain นั้นๆ ไม่มีการตัดสินใจใดๆ ตัว แพ็กเก็ต ก็ สามารถย้อนกลับมายัง กฎ ถัดไปใน chain ที่เริ่มต้นเดินทางได้ (ศึกษารายละเอียดได้จากตัวอย่าง ด้านล่าง)

INPUT	test
Rule1: -p ICMP -j DROP	Rule1: -s 192.168.1.1
Rule2: -p TCP -j test	Rule2: -d 192.168.1.1
Rule3: -p UDP -j DROP	

เช่น ถ้า ทีซีพี แพ็กเก็ต เดินทางจาก 192.168.1.1 ไปยัง 1.2.3.4 ดังนั้น แพ็กเก็ต จะเข้าสู่ INPUT chain และไม่ match กับ กฎ1 แต่ match กับ กฎ2 ซึ่งมี target เป็น test ดังนั้น แพ็กเก็ต จะเข้าสู่ test chain และ match กับ กฎ1 แต่เนื่องจาก กฎ1 ของ test ไม่ได้ระบุ target ดังนั้น แพ็กเก็ต จึงผ่านไปยัง กฎ2 ซึ่งไม่ match จากนั้น แพ็กเก็ต จึงจะเดินทางกลับไปยัง กฎ3 ของ INPUT chain อีกครั้ง ซึ่งก็ไม่ match เช่นกัน ในกรณีที่ผ่าน กฎ ทั้งหมดแล้วแต่ไม่ match หรือ match แต่ไม่มี target นั้น แพ็กเก็ต จะถูก DROP หรือ ACCEPT ก็ขึ้นอยู่กับ default policy ของ chain นั้นๆ ซึ่งสามารถตั้งค่าได้ง่ายๆ

เช่น # iptables -P INPUT DROP หรือ # iptables -P FORWARD ACCEPT

- new target

เป็น target ที่สร้างเพิ่มเติมขึ้นมาคือ

- o LOG

เป็นโมดูลที่มีความสามารถในการเก็บข้อมูลลงล็อก (มี syslog facility เป็น kernel)

สำหรับ แพ็กเก็ต ที่ match กับ กฎ ที่ระบุ target เป็น LOG มีופןให้เลือกใช้งานดังนี้คือ

- --log-level

เป็นการระบุ priority level ของ log ซึ่งกำหนดได้ตั้งแต่ debug, info, notice, warning, crit, alert, emerg รายละเอียดเกี่ยวกับ syslog สามารถอ่านได้ที่

http://thaicert.nectec.or.th/paper/unix_linux/linux_syslog.php

- --log-prefix

ตามด้วยชุดของตัวอักษรยาวไม่เกิน 29 ตัว ซึ่งชุดของตัวอักษรดังกล่าวจะปรากฏอยู่บนล็อกไฟล์

- o REJECT

คล้ายกับ DROP เพียงแต่จะส่ง ไอซีเอ็มพี port unreachable กลับไปยังผู้ที่ส่ง แพ็กเก็ต มา (ข้อยกเว้นคือ ไอซีเอ็มพี error message ไม่ response กับ ไอซีเอ็มพี error message ด้วยกันเอง เพราะอาจจะทำให้เกิดลูปที่ไม่รู้จบ) ทั้งนี้สามารถใช้ร่วมกับ --reject-with ตามด้วย argument ที่ต้องการได้ รายละเอียดโปรดศึกษาจากคู่มือการใช้งาน iptables ที่มาพร้อมตัวโปรแกรม (#man iptables)

- special built-in target

- o RETURN

กรณีที่เกิด match กับ กฎ ที่มี target เป็น RETURN นั้นเหมือนกับเป็นคำสั่งให้ออกไปจาก chain ปัจจุบัน เช่น หาก match กับ กฎ ที่อยู่ใน built-in chain (INPUT, FORWARD, OUTPUT) เกิดขึ้น ก็จะถูกโยนไปยัง default policy ของ chain นั้นๆ และหาก เกิด match กับ กฎ ที่เป็น user-defined chain ตัว เกิดขึ้น จะถูกโยนออกมา chain ก่อนหน้านั้น

- o QUEUE

เป็น chain พิเศษ ใช้สำหรับส่งต่อ เกิดขึ้น ไปยัง application ที่เขียนขึ้นมารองรับ โดยเฉพาะ โดยจะต้องมี queue handler และ application เป็นส่วนประกอบที่จะทำงานร่วมกัน

5.4 การเดินทางของ แพ็กเก็ต ในระบบ

เมื่อ แพ็กเก็ต เข้ามาถึงไฟร์วอลล์ มันจะผ่านฮาร์ดแวร์เข้ามายัง device ที่เหมาะสมในเคอร์เนล จากนั้น แพ็กเก็ต จะเดินทางไปเป็นทอดๆ ก่อนที่จะถูกส่งไปยังปลายทางที่แท้จริง เช่น แอปพลิเคชันในเครื่องไฟร์วอลล์ หรือ forward ต่อไปยังเครื่องอื่น ซึ่งจะยกตัวอย่างเพื่อให้เห็นภาพอย่างชัดเจนดังนี้

Step	Table	Chain	Comment
1			ข้อมูลอยู่ในระหว่างการเดินทาง เช่น กำลังมาจากอินเทอร์เน็ต
2			ข้อมูลเข้ามายังเครื่องไฟร์วอลล์ผ่านทาง incoming interface (เช่น eth0)
3	mangle	PREROUTING	ใช้สำหรับการทำ mangling เกิดขึ้น เท่านั้น เช่น เปลี่ยนค่า TOS ของ แพ็กเก็ต ซึ่งในกรณีปกติแล้วแทบไม่ได้ใช้งาน
4	nat	PREROUTING	chain นี้ใช้สำหรับทำ Destination Network Address Translation ไม่ควรสร้าง กฎ เพื่อกรอง เกิดขึ้น ที่ chain นี้ เพราะอาจจะมึนงง เกิดขึ้น ที่ ไม่เข้าสู่ chain นี้ (มีแค่ เกิดขึ้น แรกเท่านั้นที่ผ่านเข้าสู่ chain ส่วน เกิดขึ้น ถัดไปใน connection เดียวกันนั้น จะถูกกระทำเหมือนกับที่ เกิดขึ้น แรกได้รับ)

Step	Table	Chain	Comment
5			เข้าสู่ Routing decision เพื่อตัดสินใจว่า แพ็กเก็ต จะถูกส่งไปที่ใด
6	filter	FORWARD	เนื่องจากในตัวอย่างนี้ แพ็กเก็ต จะถูกส่งไปยังเครื่องอื่นในเครือข่าย ดังนั้น แพ็กเก็ต จึงต้องเข้า FORWARD chain ของ filter table ซึ่งสามารถเขียน กฎ สำหรับควบคุมการผ่านเข้าออกของ แพ็กเก็ต สำหรับ forwarded แพ็กเก็ต ได้ที่นี่
7	nat	POSTROUTING	และก่อนที่ แพ็กเก็ต จะออกไปจากเครื่องไฟร์วอลล์ โดยส่วนใหญ่(ไม่ใช่ทั้งหมด) จะผ่าน chain นี้ ซึ่งใช้ทำ Source Network Address Translation ไม่ควรสร้าง กฎ เพื่อกรอง แพ็กเก็ต ที่ chain นี้ เพราะอาจจะมีบาง แพ็กเก็ต ที่ไม่เข้าสู่ chain นี้ (มีแค่ แพ็กเก็ต แรกเท่านั้นที่ผ่านเข้าสู่ chain ส่วน แพ็กเก็ต ถัดไปใน connection เดียวกันนั้น จะถูกกระทำเหมือนกับที่ แพ็กเก็ต แรกได้รับ)
8			แพ็กเก็ต ออกไปทาง outgoing interface (เช่น eth1)
9			แพ็กเก็ต เดินทางไปสู่เป้าหมาย (เช่น ผ่านทาง LAN)

อย่างที่เห็นจากตาราง forwarded แพ็กเก็ต คือ แพ็กเก็ต ที่มีปลายทางที่ไม่ใช่เครื่อง ไฟร์วอลล์ อาจจะเป็น แพ็กเก็ต ที่มาจากอินเทอร์เน็ต หรืออาจจะเป็น แพ็กเก็ต ที่มาจากเครื่องลูกในเครือข่ายที่ต้องการส่งออกไ อินเทอร์เน็ต ซึ่งไม่ว่าจะส่งในทิศทางใด แพ็กเก็ต ก็จะต้องผ่าน chain ในลักษณะด้านบนนี้เสมอ

ตารางที่ 5-2 การเดินทางของฟอว์เวิร์ดแพ็กเก็ต (Forwarded แพ็กเก็ต)

Step	Table	Chain	Comment
1			ข้อมูลอยู่ในระหว่างการเดินทาง เช่น กำลังมาจากอินเทอร์เน็ต
2			ข้อมูลเข้ามายังเครื่องไฟร์วอลล์ผ่านทาง incoming interface (เช่น eth0)
3	mangle	PREROUTING	ใช้สำหรับการทำ mangling แพ็กเก็ต เท่านั้น เช่น เปลี่ยนค่า TOS ของ แพ็กเก็ต ซึ่งในกรณีปกติแล้วแทบไม่ได้ใช้งาน
4	nat	PREROUTING	chain นี้ใช้สำหรับทำ Destination Network Address Translation ไม่ควรสร้าง กฎ เพื่อกรอง แพ็กเก็ต ที่ chain นี้ เพราะอาจจะมีบาง แพ็กเก็ต ที่ไม่เข้าสู่ chain นี้ (มีแค่ แพ็กเก็ต แรกเท่านั้นที่ผ่านเข้าสู่ chain ส่วน แพ็กเก็ต ถัดไปใน connection เดียวกันนั้น จะถูกกระทำเหมือนกับที่ แพ็กเก็ต แรกได้รับ)

Step	Table	Chain	Comment
5			เข้าสู่ Routing decision เพื่อตัดสินใจว่า แพ็กเก็ต จะถูกส่งไปที่ใด
6	filter	INPUT	ทุก แพ็กเก็ต ที่มีเป้าหมายเป็นเครื่องไฟร์วอลล์จะต้องเข้าสู่ chain นี้เสมอไม่ว่าจะมาจาก interface ใดก็ตาม
7			Local process/application (เช่น server/client program)
แพ็กเก็ต ที่มีปลายทางเป็นเครื่องไฟร์วอลล์นั้น จะต้องผ่าน INPUT chain เสมอ			

ตารางที่ 5-3 การเดินทางของแพ็กเก็ตที่โฮสต์ปลายทาง (Destination localhost)

Step	Table	Chain	Comment
1			Local process/application (เช่น server/client program)
2	Mangle	OUTPUT	ใช้สำหรับการทำ mangling แพ็กเก็ต เท่านั้น การกรอง แพ็กเก็ต ที่ chain นี้จะไม่มีผลใดๆ ต่อ แพ็กเก็ต
3	Nat	OUTPUT	ไม่ได้ใช้งาน
4	Filter	OUTPUT	ใช้สำหรับกรอง แพ็กเก็ต ที่ออกมาจาก localhost หรือเครื่องไฟร์วอลล์เอง
5			เข้าสู่ Routing decision เพื่อตัดสินใจว่า แพ็กเก็ต จะถูกส่งไปที่ใด
6	Nat	POSTROUTING	และก่อนที่ แพ็กเก็ต จะออกไปจากเครื่องไฟร์วอลล์ โดยส่วนใหญ่ (ไม่ใช่ทั้งหมด) จะผ่าน chain นี้ ซึ่งใช้ทำ Source Network Address Translation ไม่ควรสร้าง กฎ เพื่อกรอง แพ็กเก็ต ที่ chain นี้ เพราะอาจจะมึบบาง แพ็กเก็ต ที่ไม่เข้าสู่ chain นี้ (มีแค่ แพ็กเก็ต แรกเท่านั้นที่ผ่านเข้าสู่ chain ส่วน แพ็กเก็ต ถัดไปใน connection เดียวกันนั้น จะถูกกระทำเหมือนกับที่ แพ็กเก็ต แรกได้รับ)
7			แพ็กเก็ต ออกไปทาง outgoing interface (เช่น eth1)
8			Local process/application (เช่น server/client program)

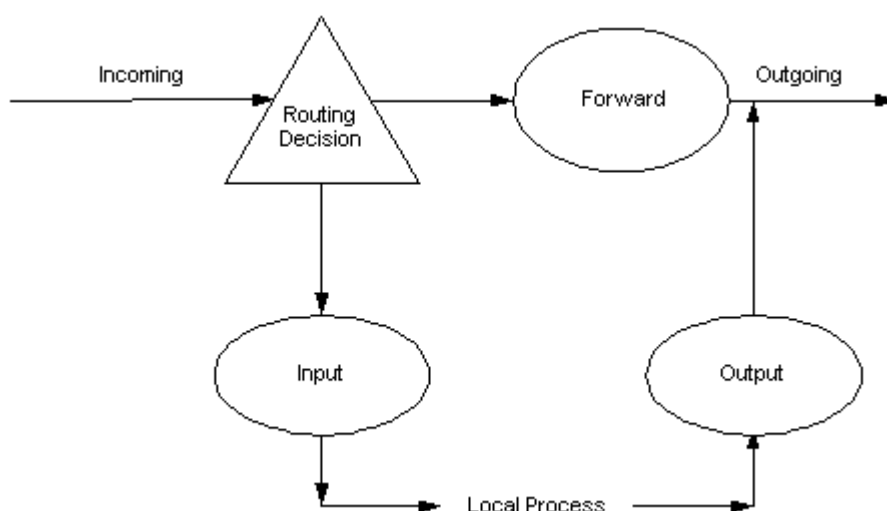
ตารางที่ 5-4 การเดินทางของแพ็กเก็ตที่โฮสต์ต้นทาง (Source localhost)

จากสามตัวอย่างด้านบน คงพอจะมองเห็นภาพออกแล้วว่า แพ็กเก็ต เดินทางเข้าออกในระบบอย่างไร อย่างไรก็ตาม จะสรุปความสำคัญของแต่ละตาราง (table) อีกครั้ง ดังนี้

5.5 ฟิลเตอร์เทเบิล (Filter Table)

เป็นตารางที่ใช้งานมากที่สุด เป็นจุดที่ใช้ในการตรวจสอบและควบคุมการผ่านเข้าออกของ แพ็กเก็ต ถ้าหากจะพิจารณาการไหลเวียนของ แพ็กเก็ต เฉพาะในส่วนของ filter table โดยไม่สนใจ table อื่นๆ นั้น ก็พอจะแสดงให้เห็นได้ดังภาพที่ 1 โดยเมื่อ แพ็กเก็ต เข้ามาในระบบ มันจะเข้าไปยัง routing decision เพื่อตัดสินใจว่า แพ็กเก็ต จะถูกส่งไปที่ใด

- ในกรณีที่ แพ็กเก็ต ถูกส่งผ่านไปยังเครื่องอื่น แพ็กเก็ต นั้นจะต้องถูกตรวจสอบโดย กฎ ใน FORWARD chain
- ถ้า แพ็กเก็ต นั้น มีเป้าหมายเป็นเครื่องปัจจุบัน (เครื่องที่รัน iptables อยู่นี้ เรียกอีกอย่างว่า ลิ้นชัก box) ตัว แพ็กเก็ต จะถูกตรวจสอบโดย กฎ ใน INPUT chain
- และในกรณีที่ แพ็กเก็ต ถูกสร้างจากเครื่องปัจจุบัน (ลิ้นชัก box) ตัว แพ็กเก็ต จะถูกตรวจสอบจาก กฎ ใน OUTPUT chain ก่อนที่จะถูกส่งออกไป



รูปที่ 5-1 แสดงให้เห็นว่า แพ็กเก็ต มีเส้นทางการเดินทางอย่างไรเมื่อเข้ามาในระบบ (filter table)

ดังภาพ iptables ประกอบไปด้วย built-in chain จำนวน 3 chain ซึ่งไม่สามารถลบได้คือ INPUT, OUTPUT, FORWARD เมื่อเครื่องคอมพิวเตอร์เริ่มทำงานในครั้งแรก ทั้งสาม chain จะมี default policy เป็น ACCEPT ซึ่งหมายความว่าอนุญาตให้ทุกอย่างผ่านเข้าออกได้หมด และสำหรับ FORWARD chain นั้น ถึงแม้จะได้กำหนดให้ policy เป็น ACCEPT แล้ว แพ็กเก็ต ก็จะไม่สามารถถูก forward ไปยังจุดหมายที่ต้องการได้ トラバドที่ยังไม่ได้เซตให้ enable IP forwarding ทั้งนี้โดย default แล้ว forward=0 สามารถกำหนดให้ enable IP forwarding (forward=1) ได้โดยใช้คำสั่ง `echo "1" > /proc/sys/net/ipv4/ip_forward` เพื่อกำหนดให้ IP forwarding เป็น enable เพื่อให้ ลิ้นชัก box สามารถ forward ip แพ็กเก็ต ได้ ในบางครั้งนั้นการใช้คำสั่งดังกล่าวทุกครั้งอาจจะไม่สะดวก สามารถแก้ไขไฟล์ configuration ที่ `/etc/sysctl.conf` แล้ว set ให้ `net.ipv4.ip_forward=1` เพื่อเป็นการแก้ไขแบบถาวร ในกรณีที่ต้องการให้

สนับสนุนการทำงานกับ dynamic IP ด้วย เช่น PPP, SLIP, DHCP ก็สามารถทำได้โดยใช้คำสั่ง `echo "1" > /proc/sys/net/ipv4/ip_dynaddr` ได้เช่นเดียวกัน

*** เนื่องจากโครงงานนี้ศึกษาเฉพาะ Filter Table จึงไม่กล่าวถึง NAT Table และ Mangle Table ในที่นี้